

🎮 Introduction to Pygame in Python

What is Pygame?

Pygame is a free and open-source Python library used to create **2D games and multimedia applications**. It provides modules for handling:

- Graphics (drawing shapes, images, sprites)
- Sound and music
- Keyboard and mouse input
- Collision detection
- Game loops and timing

Step-by-step: Installing pygame in Anaconda environment

---

1. Open Anaconda Prompt

You can find it in your Start Menu (Windows) or just open your terminal if you're on macOS/Linux and have conda installed.

---

2. Activate your environment

If you already have an environment created (let's say it's called `myenv`), activate it by running:

```
conda activate myenv
```

If you haven't created an environment yet, create one with:

```
conda create -n myenv python=3.9
conda activate myenv
```

---

3. Install pygame

You have two main options:

Option A: Install pygame via conda-forge channel (recommended)

```
conda install -c conda-forge pygame
```

This installs pygame from the `conda-forge` community channel and is usually well-maintained.

Option B: Install pygame using pip inside conda env

```
pip install pygame
```

Use this if the conda package doesn't work for you or is outdated.

---

4. Verify installation

Start Python in the same activated environment:

```
python
```

Then run:

```
import pygame
print(pygame.ver)
```

If no errors appear and you see the version info, pygame is ready to go!



This is a Github link of a recreated 'Super Mario' game:

https://github.com/justinmeister/Mario-Level-1

The complete Pygame API documentation is available at:
https://www.pygame.org/docs/

PyGame Event System Common Event Types:

pygame.QUIT: User closes window
pygame.KEYDOWN: Key pressed
pygame.KEYUP: Key released
pygame.MOUSEBUTTONDOWN: Mouse button pressed
pygame.MOUSEBUTTONUP: Mouse button released
pygame.MOUSEMOTION: Mouse moved

PyGame Drawing Functions Basic Shapes:

pygame.draw.rect() - Rectangles

```
pygame.draw.rect(surface, color, rect, width=0)
```

- **surface**: The surface to draw on (e.g., screen)
- **color**: RGB tuple (r, g, b) or color constant
- **rect**: pygame.Rect object or tuple (x, y, width, height)
- **width** (optional): Line thickness (0 = filled, >0 = outline thickness)

pygame.draw.circle() - Circles

```
pygame.draw.circle(surface, color, center, radius, width=0)
```

- **surface**: The surface to draw on
- **color**: RGB tuple (r, g, b) or color constant
- **center**: Tuple (x, y) - center point of circle
- **radius**: Integer - radius of circle
- **width** (optional): Line thickness (0 = filled, >0 = outline thickness)

pygame.draw.line() - Lines

```
pygame.draw.line(surface, color, start_pos, end_pos, width=1)
```

- **surface**: The surface to draw on
- **color**: RGB tuple (r, g, b) or color constant
- **start_pos**: Tuple (x1, y1) - starting point
- **end_pos**: Tuple (x2, y2) - ending point
- **width** (optional): Line thickness (default = 1)

pygame.draw.polygon() - Polygons

```
pygame.draw.polygon(surface, color, points, width=0)
```

- **surface**: The surface to draw on
- **color**: RGB tuple (r, g, b) or color constant
- **points**: List of tuples [(x1,y1), (x2,y2), ...] - vertices of polygon
- **width** (optional): Line thickness (0 = filled, >0 = outline thickness)

pygame.draw.ellipse() – Ellipses

```
pygame.draw.ellipse(surface, color, rect, width=0)
```

- **surface**: The surface to draw on
- **color**: RGB tuple (r, g, b) or color constant
- **rect**: pygame.Rect object or tuple (x, y, width, height) - bounding rectangle
- **width** (optional): Line thickness (0 = filled, >0 = outline thickness)

Coordinate System:

(0,0)

+----------------------> x (right)

|

|

|

v y (down)

Top-left corner is (0,0) X increases rightward Y increases downward

Benefits of Using Clock

1. **Consistent Speed**: Game runs same speed on all computers
2. **Smooth Animation**: Prevents jerky movement
3. **CPU Friendly**: Doesn't use 100% of CPU
4. **Predictable Timing**: Easy to control game timing

Without clock: Fast computers = super fast game, Slow computers = super slow game

With clock: All computers = same game speed

```
clock = pygame.time.Clock()  # Create a clock object
```

Fonts:

`pygame.font.Font(filename, size):` **create a font**

- **filename**: Path to font file, or `None` for default font
- **size**: Font size in pixels

`font.render(text, antialias, color,background=None) :`

Turns text into a drawable image (surface)

- **text**: String to display
- **antialias**: `True` for smooth edges, `False` for pixelated
- **color**: Text color (RGB tuple)
- **background**: Optional background color (RGB tuple)

`screen.blit(surface, position) :` **Draws the text surface onto the screen**

- **surface**: The text surface to draw
- **position**: (x, y) coordinates where to draw

Exercise 1: Arrow Key Rectangle Mover

Objective: Create a Pygame program where you can move a rectangle around the screen using the arrow keys.

Requirements:

- Create a 800x600 window
- Draw a red rectangle (80X50 pixels) that starts in the center of the screen
- Use the arrow keys to move the rectangle:
    - UP arrow: move up
    - DOWN arrow: move down
    - LEFT arrow: move left
    - RIGHT arrow: move right
- The rectangle should move smoothly and not leave the screen boundaries

Bonus Challenges:

1. Add different movement speeds for normal movement vs. holding Shift for faster movement
2. Make the rectangle change color when it hits the screen edges
3. Add a reset function that returns the rectangle to center when pressing Spacebar

Exercise 2: Mouse-Following Circle & Click Effects

Objective: Create a Pygame program that responds to mouse movements and clicks with visual feedback.

Requirements:

- Create a 800x600 window
- Draw a circle that follows the mouse cursor position
- Implement these mouse interactions:
- **Mouse Movement:** A blue circle follows the cursor
    - **Left Click:** Create a red circle at click position that stays on screen
    - **Right Click:** Create a green circle at click position that stays on screen
    - **Mouse Wheel Scroll:** Change the size of the following circle
- Keep all clicked circles visible until the program closes

Bonus Challenges:

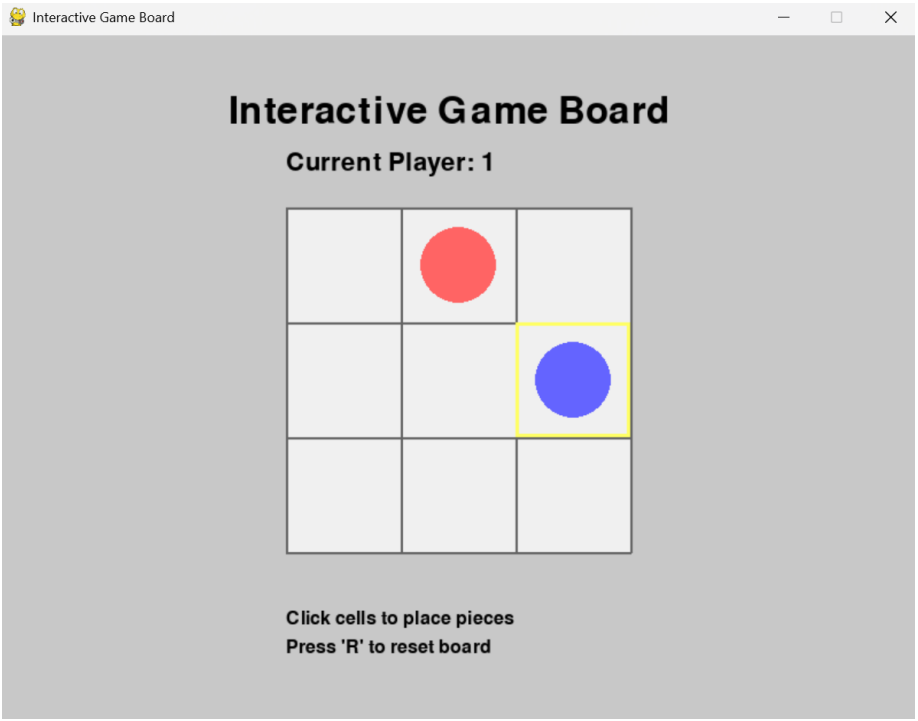
1. Add middle mouse button click to clear all drawn circles
2. Make the circles fade out slowly over time
3. Add different shapes for different mouse buttons (squares, triangles)

Exercise 3: Tic-Tac-Toe Game Implementation

Build a Simple Game Board.

Features:

- Grid representation
- Cell selection
- Visual feedback
- State management



Sultan Qaboos University

Dr. Noushath Shaffi

Teacher Assistant : Shahad Ali Al Sulaimani: s142327@student.squ.edu.om